

Assistente de IA para documentação de logística com RAG ✓

```
novatech_rag_co  
ntext_engineerin  
g.md
```

MD

O Cenário A NovaTech é uma empresa de médio porte do setor de logística com 1.200 funcionários. Sua operação depende de um conjunto extenso de documentação interna: manuais de procedimento operacional, políticas de compliance, tabelas de SLA por tipo de cliente, regras de cálculo de frete, e normas de segurança de carga. Hoje, essa documentação está espalhada em três fontes: um SharePoint corporativo com ~800 documentos (PDFs e Word), uma wiki interna no Confluence

[Mostrar mais](#)

Preparou-se para elaborar análise técnica detalhada >

Lido o documento completo. Aqui está a revisão técnica detalhada:

Avaliação Geral

A análise tem boa qualidade técnica: identifica os desafios reais de cada fonte, propõe ferramentas pertinentes e demonstra familiaridade com conceitos de RAG. Mas há erros técnicos pontuais, premissas não fundamentadas e riscos críticos ausentes que podem comprometer o projeto em produção.

1. Erros Técnicos e Informações Incorretas

`openpyxl` com `data_only=True` — bug crítico não documentado

Este é o erro mais grave da análise. O `data_only=True` do `openpyxl` não recalcula fórmulas: ele lê os valores em cache salvos pela última vez que o arquivo foi aberto no Microsoft Excel. Se as planilhas da NovaTech são geradas ou modificadas por scripts Python, automações ou qualquer ferramenta que não seja o Excel nativo, o cache não

existe — e `data_only=True` retorna `None` para todas as células com fórmula, silenciosamente, sem erro.

```
python

# Resultado real em planilha salva por script:
ws['C5'].value # → None, não R$ 48,00
```

A solução correta depende do ambiente: `xlwings` no Windows (requer Excel instalado), conversão prévia via LibreOffice com `--headless --calc`, ou reformulação para que as planilhas sejam salvas com valores pré-calculados antes da ingestão no pipeline.

Regra de tokenização subestimada para português

A regra "1 token $\approx$ 0,75 palavras" é calibrada para inglês. Para português, com morfologia mais rica (conjugações, sufixos, palavras compostas), a relação empírica é de aproximadamente 1 token $\approx$ 0,60–0,65 palavras nos tokenizadores baseados em BPE (GPT, Claude). Isso significa que a estimativa total de $\sim$ 4,2M tokens está **subestimada em 15–25%** — a base real provavelmente está entre 5,0M e 5,3M tokens. A conclusão (RAG é necessário) permanece válida, mas o orçamento de embedding e custo de ingestão precisam da correção.

Afirmção sobre Claude e Lost in the Middle sem evidência sólida

O documento afirma que "modelos Claude são treinados para maior recall em posições intermediárias do contexto, atenuando o Lost in the Middle." Não há paper ou benchmark público da Anthropic que sustente essa afirmação de forma conclusiva para contextos de 100K+ tokens. Pode ser verdade em graus, mas a recomendação de `k ≤ 15` deve ser mantida para qualquer modelo, independentemente dessa atribuição.

2. O Orçamento de Contexto Demonstra Compreensão da Context Window?

Parcialmente. O conceito está correto — a context window é limitada, o overhead consome parte dela, e o restante é o espaço disponível para chunks. Mas há dois problemas conceituais relevantes:

Conforme o documento

GPT-4o — 128K tokens

System prompt

~2.000 tokens

Query do usuário

~200 tokens

Buffer de resposta gerada **problema**

~2.000 tokens — subtraído da janela de input

Histórico de conversa

— não mencionado

Disponível para chunks

123.800 tokens → ~247 chunks teóricos

✗ Output tokens são tratados como se consumissem o mesmo orçamento que os tokens de input.

Modelo correto

GPT-4o — janela de input + output separados

System prompt **revisar**

~3.000–6.000 tokens (prompt enterprise elaborado)

Query do usuário

~200 tokens

Histórico de conversa **ausente**

~3.000–8.000 tokens (3–5 turnos de chat)

Disponível para chunks

~114.000–124.000 tokens → ~228–248 chunks teóricos

↑ separado — output tokens não consomem o input budget

Tokens de output (resposta)

até 16.384 tokens (GPT-4o) — orçamento próprio

✓ O impacto prático nos chunks disponíveis é pequeno, mas o framing conceitual incorreto pode gerar confusão em contextos com histórico longo.

Problema 1 — O "buffer de resposta" no orçamento de input

Para GPT-4o e Claude, os tokens de output não consomem o budget da janela de input — são orçamentos separados (GPT-4o suporta até 16K de output; Claude até 8K).

Subtrair 2.000 tokens de "buffer de resposta" da janela de input é conceitualmente incorreto para esses modelos. O impacto numérico é pequeno, mas o framing pode confundir o time de implementação sobre como dimensionar `max_tokens` vs. tamanho do prompt.

Problema 2 — Histórico de conversa ausente do overhead

O overhead calculado (4.200 tokens) ignora completamente o histórico da conversa. Em um assistente de atendimento com interface de chat, é esperado que cada turno seja adicionado ao contexto. Em uma conversa de 4 turnos com respostas moderadas, o histórico acrescenta facilmente 5.000–8.000 tokens ao overhead — reduzindo o espaço real para chunks e podendo empurrar o número prático de `k` para 5–8, não 10–15.

O que está correto: A recomendação prática de `k = 5–15` está bem fundamentada, e o posicionamento de chunks relevantes no início do contexto (primacy effect) é tecnicamente sólido.

3. A Estimativa de Tokens é Razoável?

A ordem de grandeza é razoável, mas há três premissas não fundamentadas que precisam de revisão antes de usar o número em qualquer planejamento de custo:

A média de 10 páginas por documento PDF foi assumida sem base — não está no briefing. Documentos de logística variam de 1 página (tabela de SLA resumida) a 80+ páginas (manual operacional completo). Uma amostra de 20 documentos reais seria suficiente para calibrar essa média antes do início do projeto.

O número de 50 arquivos de planilha também foi assumido livremente. O cenário menciona apenas "pasta de rede com planilhas atualizadas mensalmente", sem quantidade.

A serialização contextual das tabelas — que transforma cada linha em texto autoexplicativo — adiciona overhead considerável de tokens que não está contabilizado. Uma tabela de 100 linhas × 15 colunas com headers replicados em cada chunk pode gerar 1,5× a 2× o volume de tokens do conteúdo bruto. O total real das planilhas pode ser 300K-400K tokens, não 200K.

4. Estimativas Otimistas e Riscos Não Considerados

Prazo de 3 meses é muito apertado para o escopo descrito

O escopo real acumula pelo menos 6 pipelines de ingestão distintos: PDFs nativos com tabelas via Document Intelligence, PDFs escaneados via OCR + normalização, fluxogramas via Vision API, Confluence com spider de links e remoção de macros, planilhas com serialização contextual e resolução de dependências entre abas, e Text-to-SQL como estratégia paralela. Acrescentando reranking com cross-encoder, HyDE para queries abertas, metadados de versão e controle de mudanças por hash, o escopo real equivale a 5-6 meses de desenvolvimento competente. Discovery + dev + go-live em 3 meses exige escopo prioritizável e roadmap explícito com MVP reduzido.

Contradições documentais: o problema central não está resolvido

O cenário descreve explicitamente documentos que se contradizem e ausência de processo unificado de revisão. A análise propõe metadados de `data_publicacao` e `versao` para priorizar a versão mais recente — mas não aborda o fato de que esses metadados precisam existir de forma confiável nos documentos originais. Se um PDF escaneado de 2019 não tem data legível e uma versão Word de 2023 não tem campo de versão preenchido, o filtro por data falha silenciosamente. O assistente pode servir a versão desatualizada com aparência de confiança. Esse risco de negócio não está endereçado.

Sem estratégia de avaliação

Nenhuma menção a como medir se o sistema atingiu o objetivo de negócio (reduzir de 12 para menos de 2 minutos por chamado). Um pipeline RAG enterprise precisa de métricas desde o início: precision@k, recall@k, faithfulness (a resposta é fiel aos chunks recuperados?), latência end-to-end. Sem isso, não é possível iterar com evidência nem demonstrar sucesso à diretoria.

Modelo de embedding não discutido

A seleção do modelo de embedding é crítica para português e está completamente ausente. `text-embedding-ada-002` e `text-embedding-3-large` têm performance materialmente inferior em português comparados com modelos multilínguais como `multilingual-e5-large` ou `text-embedding-3-large` com instruções explícitas de idioma. A escolha errada aqui degrada o recall antes mesmo de qualquer otimização de chunking.

HyDE tem custo operacional não mencionado

HyDE requer uma chamada LLM adicional por query para gerar o "documento hipotético". Com 192 queries/dia envolvendo consulta a documentação, isso representa ~6.000 chamadas LLM extras por mês, com latência adicionada de 1-3 segundos por query. O trade-off vale para queries abertas complexas, mas precisa ser explicado e mensurado.

Controle de acesso

Nenhuma menção sobre permissões de documentos. O SharePoint corporativo provavelmente tem ACLs — documentos de compliance ou RH podem não ser acessíveis a todos os 45 atendentes. O RAG precisa respeitar as permissões da fonte ou o assistente pode surfar informação restrita.

5. Melhorias Gerais

Chunk overlap ausente. A análise não menciona sobreposição entre chunks, que é prática padrão para evitar perda de contexto nas bordas. Um overlap de 10-20% (ex.: 50-100 tokens) entre chunks adjacentes é especialmente importante para os chunks de política (seções semânticas), onde a regra relevante pode estar exatamente na fronteira entre dois chunks. Essa omissão é significativa.

Inconsistência entre seções. A Seção 1.3 propõe incluir "os primeiros 200 tokens das páginas linkadas" no Confluence como contexto adicional. A Seção 4 recomenda chunks de 400-500 tokens para perguntas de política/regra. Os 200 tokens de contexto de link são muito curtos para ser úteis em queries de política e provavelmente

capturarão só o cabeçalho da página linkada. Faz mais sentido indexar as páginas linkadas como chunks completos e recuperá-las via retrieval, não como prefixo fixo.

Text-to-SQL está subestimado como esforço. É listado como "fallback" e "estratégia paralela" para tabelas e planilhas, mas Text-to-SQL é um sistema separado com requisitos próprios: schema discovery, prompt engineering específico, validação de queries geradas, tratamento de joins entre tabelas e gestão de erros semânticos. O esforço de implementação é equivalente a um sub-projeto independente.

A heurística de detecção de PDFs escaneados precisa ser refinada. Usar "menos de 10% das páginas com texto nativo" como limiar de acionamento do OCR trata o documento como homogêneo. Documentos híbridos (páginas com texto + páginas escaneadas) são comuns em arquivos corporativos legados. A detecção deve ser feita por página, não por documento.

Pontos bem fundamentados que devem ser preservados: A serialização contextual de tabelas linha a linha é a abordagem correta e frequentemente subestimada. O breadcrumb como metadado de hierarquia no Confluence é uma escolha madura. A estratégia de confidence score como metadado de alerta para OCR de baixa qualidade é elegante. O hash SHA-256 para detecção de mudanças nas planilhas é simples e eficaz.

Implemente essas melhorias no documento

Redigindo documento técnico com melhorias implementadas >

Documento atualizado. Aqui está um resumo de cada mudança implementada, organizado por seção:

Seção 1 — Fontes de dados

- 1.1: adicionado aviso de que Text-to-SQL é um sub-projeto independente de esforço significativo, não um "fallback trivial"
- 1.2: corrigida a detecção de PDFs escaneados de nível de documento para nível de página individual, com tratamento explícito de documentos híbridos
- 1.3: removida a abordagem dos "primeiros 200 tokens" de páginas linkadas; substituída por indexação completa com metadado `linked_from` e recuperação via retrieval normal
- 1.4: adicionado aviso crítico do bug do `openpyxl data_only=True` com três alternativas concretas (LibreOffice headless, xlwings, processo operacional)

Seção 2 — Estimativa de tokens

- Regra de tokenização corrigida de 0,75 para 0,62 palavras/token para português
- Adicionado fator de overhead de serialização ($\times 1,60$) para planilhas
- Todos os cálculos refeitos: total sobe de ~4,2M para ~5,2M tokens (+24%)
- Tabela expandida com coluna de "Palavras Efetivas" para deixar o ajuste de serialização explícito

Seção 3 — Orçamento de contexto

- Removido o "buffer de resposta gerada" do budget de input — output tem orçamento separado nos modelos listados
- Sistema de prompt revisado de 2.000 para ~3.500 tokens (estimativa enterprise realista)
- Adicionado histórico de conversa ao overhead (0–6.000 tokens variáveis)
- Todos os cálculos agora têm dois cenários: sem histórico e com 5 turnos de conversa
- Removida a afirmação não comprovada sobre Claude "atenuar" o Lost in the Middle

Seção 4 — Chunking

- Adicionada política de overlap de 15–20% para todos os chunks de texto corrido, com exceção explícita para linhas de tabela serializada
- Adicionado custo operacional do HyDE (~6.000 chamadas LLM extras/mês)
- Referência à estratégia de resolução de links do Confluence alinhada com a correção da Seção 1.3

Seção 5 — Nova seção: Riscos Operacionais

- 5.1: alerta de prazo com recomendação de MVP explícito
- 5.2: estratégia de avaliação (RAGAS, Precision@k, Faithfulness, eval set de 100–200 pares)
- 5.3: seleção de modelo de embedding para português com tabela comparativa
- 5.4: confiabilidade de metadados de versão para documentos contraditórios
- 5.5: controle de acesso e riscos de LGPD



Novatech rag context engineering v2
Documento · MD

Baixar

